Software Requirements Specification
for

# HomeAI Cloud

Intelligent Cloud Platform for Smart Homes

Version 1.0

Deliverable #1 · Project Option #1

Prepared by

| Member | Name | Component owned |
|---|---|---|
| Member 1 | Ekant Kapgate | C1 Home-Based IoT Device Manager |
| Member 2 | Renuka Prasad Patwari | C3 System Database Management |
| Member 3 | Prathamesh Ravindra Sawant | C2 Alert Tracking and Monitor |
| Member 4 | Mohit Manoj Barade | C4 System Dashboard |

San José State University
CMPE 281 Cloud Technologies · Instructor: Jerry Gao, Ph.D.
October 2026

# Revision History

| Name | Date | Reason for changes | Version |
|---|---|---|---|
| HomeAI Cloud team | Oct 2026 | Initial Deliverable #1 release, prepared on the IEEE SRS template | 1.0 |

# Contents

**Revision History** **i**

**1 Introduction** **1**
1.1 Purpose . . . 1
1.2 Document Conventions . . . 1
1.3 Intended Audience and Reading Suggestions . . . 1
1.4 Product Scope . . . 1
1.5 References . . . 2

**2 Overall Description** **2**
2.1 Product Perspective . . . 2
2.2 Product Functions . . . 2
2.3 User Classes and Characteristics . . . 3
2.4 Operating Environment . . . 4
2.5 Design and Implementation Constraints . . . 4
2.6 Assumptions and Dependencies . . . 4

**3 External Interface Requirements** **4**
3.1 User Interfaces . . . 4
3.2 Hardware Interfaces . . . 7
3.3 Software Interfaces . . . 7
3.4 Communications Interfaces . . . 7

**4 System Features** **9**
4.1 C1 Home-Based IoT Device Manager . . . 10
4.2 C2 Alert Tracking and Monitor . . . 10
4.3 C3 System Database Management . . . 11
4.4 C4 System Dashboard . . . 11
4.5 System process flow . . . 12

**5 Other Nonfunctional Requirements** **12**
5.1 Performance Requirements . . . 12
5.2 Safety Requirements . . . 12
5.3 Security Requirements . . . 12
5.4 Software Quality Attributes . . . 13
5.5 Business Rules . . . 13

**6 System Architecture Design** **13**
6.1 Cloud-Based System Infrastructure . . . 13
6.2 Technology Selection . . . 14

6.3 Component-Oriented Function Architecture . . . . . 14

**7 System Database and Cloud Design** **15**
7.1 Relational Database Design . . . . . 15
7.2 NoSQL Database Design . . . . . 18
7.3 Cloud Design: Scalability, Load Balancing and Multi-Tenancy . . . . . 19

**8 Project Development Plan and Schedule** **21**
8.1 Project Team and Roles . . . . . 21
8.2 Project Schedule . . . . . 21

**Appendix A: Glossary** **22**

**Appendix B: Analysis Models** **23**

**Appendix C: To Be Determined List** **23**

# 1 Introduction

## 1.1 Purpose

This document is the Software Requirements Specification (SRS) for HomeAI Cloud, version 1.0, an intelligent cloud platform for senior smart homes. It specifies the complete system: its requirements, users, infrastructure and architecture, database and cloud design, component interactions, and the project plan. HomeAI Cloud is deployed on Amazon EC2. Camera feeds in each room of a senior house send short audio and video clips to the cloud, a pre-trained machine learning model labels each clip from its sound (a video clip is classified on its audio track), rules turn labels into alerts, one role-based dashboard delivers the result to house owners, IoT staff and cloud service staff, and external alert recipients are notified by email.

## 1.2 Document Conventions

Functional requirements use the identifier `FR-Cx-nn`, where `Cx` is the owning component (C1 to C4) and *nn* is a sequence number. Non-functional requirements use `NFR-nn`. Use cases use `UC-nn`. Each requirement carries its own priority; a higher-level priority is not inherited by detailed requirements. Priority has two values: **Must** means the capability is needed for the integrated demo in Deliverable #4, and **Should** means it is designed now and built if time allows. Field names, table names, status values and endpoints are set in a `monospace` font.

## 1.3 Intended Audience and Reading Suggestions

The document is written for the course instructor and graders, the four developers who each own one component, and any reviewer assessing the design. Readers who want the big picture should read Section 1 and Section 2. Readers assessing requirements should read Section 4 (features) and Section 5 (non-functional requirements). Readers assessing the design should read Section 3 (external interfaces, including the user interface and communication design), Section 6 (architecture), and Section 7 (database and cloud design). Section 8 holds the project plan.

## 1.4 Product Scope

HomeAI Cloud improves the safety of seniors who live alone by turning each home's sounds into timely alerts. A home's sounds carry most of what matters after a fall, a kitchen fire or a break-in: a smoke alarm, breaking glass, a scream, a cough, a dog barking, or a television left on all night. The platform applies that idea to individual senior houses inside a senior-care community, extending the edge-to-cloud smart-service and AI cloud platforms developed in earlier work [1], [2], [3], [4]. Each clip moves along one path: camera feed → AI analysis → alert → dashboard. The product has six objectives:

1. Collect each house's audio and video data and learn the resident's daily life patterns by room and hour.
2. Detect and classify emergency, safety and security sounds such as smoke alarms, glass breaking, screams and thuds.
3. Detect and recognise resident and animal sounds such as speech, coughing, snoring, dogs and cats.
4. Generate alerts for family members, security staff and emergency handlers, with acknowledgement and resolution.
5. Serve three user groups, house owners, IoT staff and cloud staff, through role-based dashboards and statistics reports.
6. Run on Amazon EC2, with a documented design for load balancing, scalability and multi-tenancy.

The expected outcomes for Deliverable #1 and the integrated demo are:

| Outcome | Owner | How we will show it |
|---|---|---|
| Camera feed and house manager | C1 | A demo house with rooms and registered feeds that accept clip uploads and show feed status |
| AI classification and database services | C3 | An uploaded clip returns labels with confidence, for example `smoke_alarm 0.94`, stored against the clip record and visible in the record browser |
| Alert service with rules, lifecycle and recipients | C2 | A detected `distress_sound` raises a critical alert, emails the ordered emergency contacts and moves through acknowledge and resolve |
| Role-based cloud dashboard | C4 | Three separate workspaces with live alerts and platform statistics |
| Integrated deployment | All | A public HTTPS endpoint on Amazon EC2 running PostgreSQL and MongoDB |

### 1.5 References

[1] H. Begur, M. Dhawade, N. Gaur, P. Dureja, J. Gao, M. Mahmoud, J. Huang, S. Chen, and X. Ding, "An edge-based smart mobile service system for illegal dumping detection and monitoring in San Jose," in *Proc. IEEE SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI*, San Francisco, CA, USA, Aug. 2017, pp. 1–6, doi: 10.1109/UIC-ATC.2017.8397575.

[2] T. S. Goruganthu, H. Marepalli, S. Bheemineni, R. C. Bobba, D. Wang, S. Richhariya, J. Gao, and J. Liu, "Building an AI-based surveillance drone cloud platform," in *Proc. IEEE 1st Int. Workshop Future Intell. Technol. Young Researchers (FITYR)*, Jul. 2024, pp. 1–10, doi: 10.1109/FITYR63263.2024.00008.

[3] S. Balivada, J. Gao, Y. Sha, M. Lagisetty, and D. Vichare, "UAV-based transport management for smart cities using machine learning," *Smart Cities*, vol. 8, no. 5, Art. no. 154, Sep. 2025, doi: 10.3390/smartcities8050154.

[4] Y. Mehta, V. Lo, V. Mehta, K. Agarwal, C. T. Madabathula, and J. Gao, "Green energy AI cloud: Intelligent community-based electricity management system," in *Proc. IEEE Int. Conf. Energy Internet (ICEI)*, 2024, pp. 343–348, doi: 10.1109/ICEI63732.2024.10917102.

[5] J. Z. Gao, C. Chen, Y. Toyoshima, and D. K. Leung, "Engineering on the Internet for global software production," *Computer*, vol. 32, no. 5, pp. 38–47, May 1999, doi: 10.1109/2.762791.

[6] Office of the Government Chief Information Officer, "Effective Systems Analysis and Design Guide, Appendix C," OGCIO, Government of the Hong Kong SAR. [Online]. Available: https://www.ogcio.gov.hk/en/infrastructure/methodology/system_development/doc/G61c_Effective_SAnD_Guide_Appendix_C_v1_1.pdf [Accessed: Oct. 4, 2026].

[7] "YAMNet: sound event classification with an AudioSet-trained model of 521 classes," TensorFlow Hub. [Online]. Available: https://www.tensorflow.org/hub/tutorials/yamnet [Accessed: Oct. 4, 2026].

[8] "Smart Home Senior Care Platform (CMPE 281)," GitHub repository. [Online]. Available: https://github.com/neeraja0312/smart-home-senior-care-cmpe-281 [Accessed: Oct. 4, 2026].

## 2 Overall Description

### 2.1 Product Perspective

HomeAI Cloud is a new, self-contained cloud platform, not a replacement for or an add-on to an existing product. It sits between the camera feeds in a senior house and the people who respond to alerts. It follows the cloud-platform pattern of the AI surveillance drone cloud [2] and the community-scale structure of the green energy AI cloud [4], and it takes the senior-care problem framing from an earlier CMPE 281 project [8]; the requirements, design and code in this document are the team's own. Figure 1 shows the platform as the three user groups see it, with no cloud service names: camera feeds in each room send clips to HomeAI Cloud, where the four components and the AI inference service turn them into alerts; the dashboard serves the three user groups, and the alert recipients receive email notifications. The detailed, AWS-level view of the same platform is given in Section 6.1.

### 2.2 Product Functions

The platform provides four groups of functions, one per component:

- **C1 Home-Based IoT Device Manager** registers and configures camera feeds, models houses and

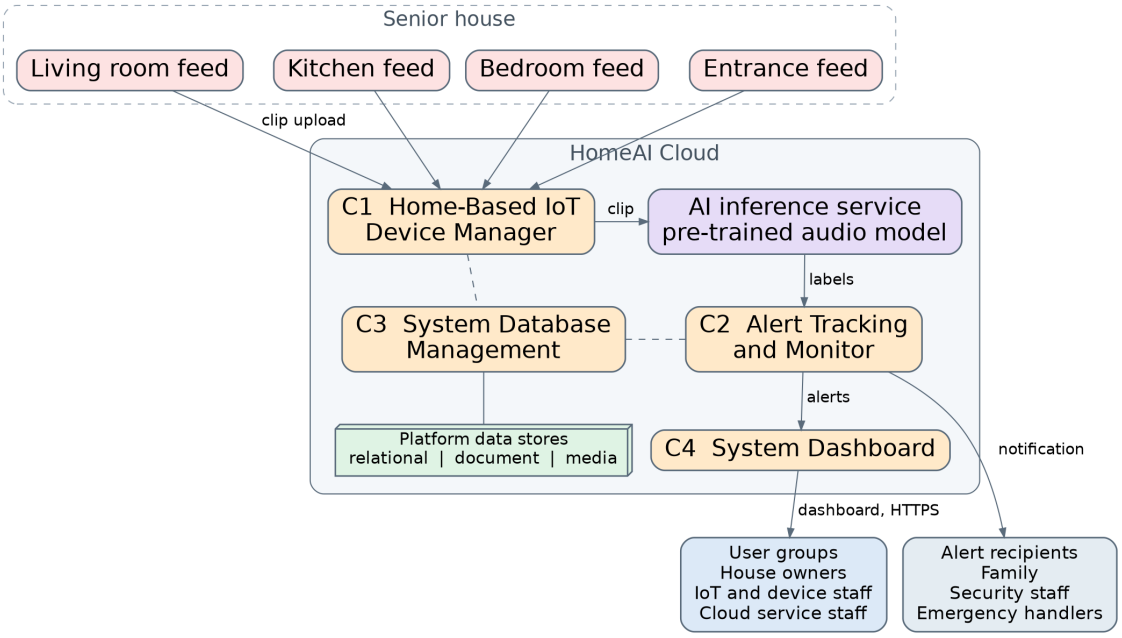



**Figure 1:** High-level business-oriented view: user groups, senior-house camera feeds, the four platform components and the alert recipients

rooms, ingests clips, and manages each house's clip records.

- **C2 Alert Tracking and Monitor** defines alert rules, evaluates each detection, raises and tracks alerts through their lifecycle, and notifies emergency contacts.
- **C3 System Database Management** runs the AI inference sub-service, maintains the data dictionary, and provides record browsing, query and integrity services across the platform data.
- **C4 System Dashboard** authenticates users, enforces role-based access control, holds owner service settings and contacts, and presents the three role workspaces and the statistics views.

The function-oriented tree and the functional block diagram that detail these groups and their subfunctions are given in Section 6.3.

## 2.3 User Classes and Characteristics

Three user classes hold accounts and sign in; a fourth group is notified without an account. Role-based access control checks the caller's role on every API call.

| User class | Description | Key functions | Data access scope |
|---|---|---|---|
| House owner `house_owner` | Senior resident or family caregiver who holds the house subscription | Home overview and live alerts; acknowledge, resolve and mark false positive; detection services, quiet hours and sensitivity; emergency contacts; per-house clip records | Own houses only, from the `house_ids` token claim |
| IoT staff `iot_staff` | The tenant's camera-feed team | Add, update, configure, enable, disable and delete camera feeds; assign a feed to a house and room; record connectivity status; upload clips | All houses of their own tenant |
| Cloud staff `cloud_staff` | Platform operators with administrative rights | Platform statistics; alert rules; record browsing, data dictionary and integrity reports across all houses | All tenants |

**External recipients.** Family members, security staff and emergency handlers are named on a house's ordered emergency-contact list. They hold no account, so when an alert is raised C2 sends each of them an email through Amazon SNS, in priority order. The email carries a signed, time-limited link that opens a read-only view of that one alert, with no login. The most important user class to satisfy is the house owner, who acts on alerts; the IoT and cloud staff classes support that outcome.

### 2.4 Operating Environment

The platform runs on one Amazon EC2 instance (t3.large) in AWS region us-west-2, with all services as Docker containers managed by Docker Compose. The server stack is Python 3.11 with FastAPI. Data is held in PostgreSQL 16, MongoDB 7 and an Amazon S3 bucket, and alert email is sent through Amazon SNS. The client is a React single-page application that runs in a current desktop or mobile web browser; no native application is required. All traffic uses HTTPS on port 443, terminated by a reverse proxy on the instance. The full technology selection is given in Section 6.2.

### 2.5 Design and Implementation Constraints

- **EC2 deployment.** The course requires Amazon EC2. The demo runs all containers on one instance with Docker Compose; the scaled layout in Section 7.3 uses the same images.
- **Camera feeds.** Feeds are software-registered media sources fed by recorded clips or a phone camera. No physical hardware, gateway or firmware exists, so connectivity is a recorded status rather than a measured radio metric.
- **Pre-trained model.** The audio model, YAMNet [7], is used as delivered. The team maps its source classes to home labels and tunes confidence thresholds; no model is trained from scratch. Video clips are classified on their audio track; no visual model is used.
- **Course timeline.** Deliverable #2 is due 16 October 2026, #3 on 6 November, #4 on 2 December and #5 on 16 December. Must requirements are sized to fit Deliverables #3 and #4.
- **Scope limits.** One AWS region and an English-only dashboard. Signed-in users are notified in the app; external recipients are notified by email only (no SMS or push).

### 2.6 Assumptions and Dependencies

The platform assumes that a usable pre-trained audio classifier is available and that its source classes can be mapped to the home alert labels; the design depends on this third-party model. It assumes that clips are supplied as recorded audio or video files, or captured by a phone camera, and uploaded over the network, since no IoT hardware is provided. It depends on the managed AWS services named in the operating environment (EC2, S3, SNS, CloudWatch, IAM) and on the PostgreSQL, MongoDB, FastAPI and React open-source stacks. If the pre-trained model proves too slow on CPU, the clip window and the set of enabled detection services are reduced; this is tracked as a project risk in Section 8.2.

## 3 External Interface Requirements

### 3.1 User Interfaces

The dashboard is a React single-page application with three role workspaces under the route prefixes `/home/*`, `/iot/*` and `/ops/*`. A route guard reads the role claim before rendering and returns a user who opens another role's URL to their own landing page; the server still enforces access control on every call. Severity and feed status are always shown with an icon and a word, never by colour alone, so the interface does not rely on colour vision.

### 3.1.1 Page and frame design

Twelve pages across the three roles, with the function buttons each one provides.

| Page | Role | Purpose | Function buttons |
|---|---|---|---|
| Login | all | Sign in and route by role | Sign in |
| Home Overview | owner | One-screen status of a house | Acknowledge, View all alerts, Switch house |
| Alerts | owner | Find and act on past and open alerts | Filter, Open, Acknowledge |
| Alert Detail | owner, cloud | Decide on one alert | Acknowledge, Resolve, Mark false positive, Play clip |
| My Feeds | owner | Check feed health, read only | Filter by room |
| Services and Settings | owner | Choose detection services | Toggle service, Set quiet hours, Set sensitivity, Save |
| Emergency Contacts | owner | Who is notified, in what order | Add, Edit, Delete, Move up, Move down |
| Camera Feed Manager | IoT | Manage the tenant's feeds | Add feed, Edit, Configure, Assign to room, Enable, Disable, Delete |
| Feed Detail | IoT | Configure and diagnose one feed | Save config, Restore version, Enable, Disable, View recent clips |
| Houses and Rooms | IoT | Model houses and their rooms | Add house, Add room, Edit, Delete |
| System Dashboard | cloud | Platform statistics at a glance | House filter, Time range, Open alert, Edit rules |
| Database Records | cloud | Browse and verify platform data | Pick entity, Filter, Page, Run integrity report, Export dictionary |

Two list views are reached from these pages rather than counted as pages: **Clip Library** (IoT), opened by View recent clips on Feed Detail and pinned in the IoT sidebar, and **Alert Rules** (cloud), opened by Edit rules on the System Dashboard. External recipients open one read-only **Alert View** from the link in their email; it needs no login and sits outside the three role workspaces. The wireframes show a feed's connectivity as online or offline, read from `connectivity_status`, which is separate from its lifecycle `status`.

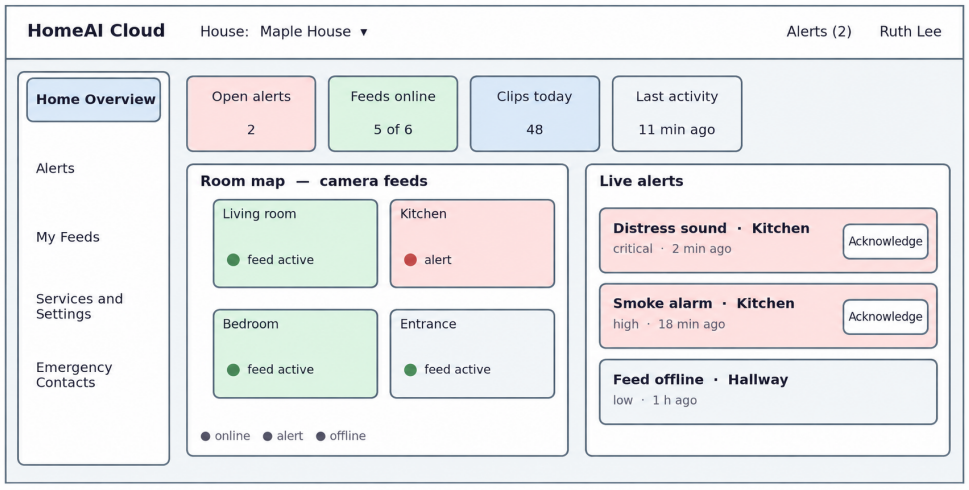



**Figure 2:** Frame 1: Home Overview, the house-owner landing page

### 3.1.2 Navigation flow

Every session starts at Login and branches once, on the role claim, into one of three workspaces. Each workspace is flat: every page is one click from the sidebar, and detail frames sit one level below their page, so no frame is more than two clicks from a landing page.

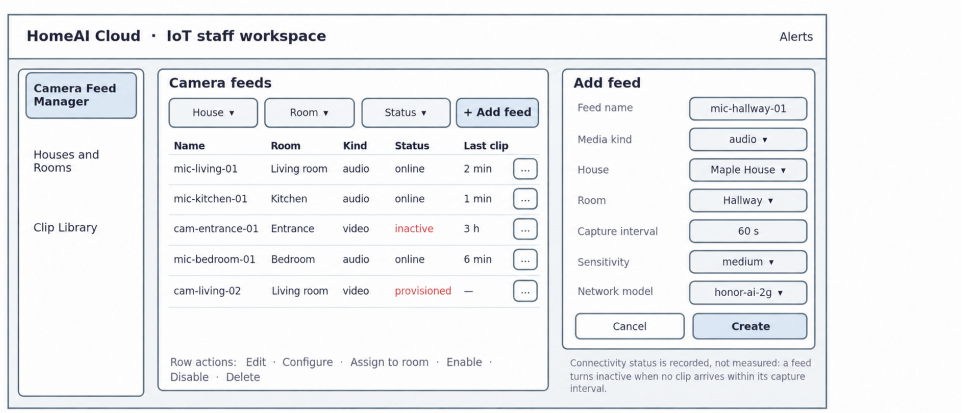



**Figure 3:** Frame 2: Camera Feed Manager with the Add feed dialog, the IoT-staff landing page

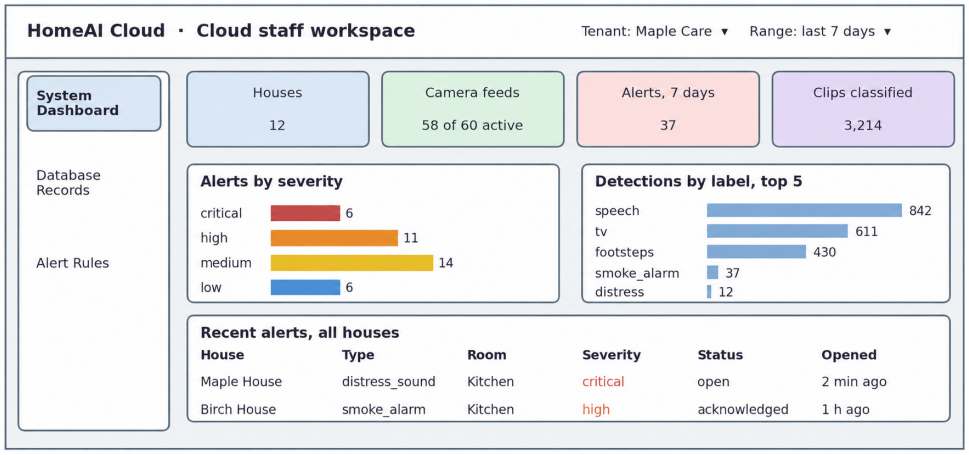



**Figure 4:** Frame 3: System Dashboard, the cloud-staff landing page

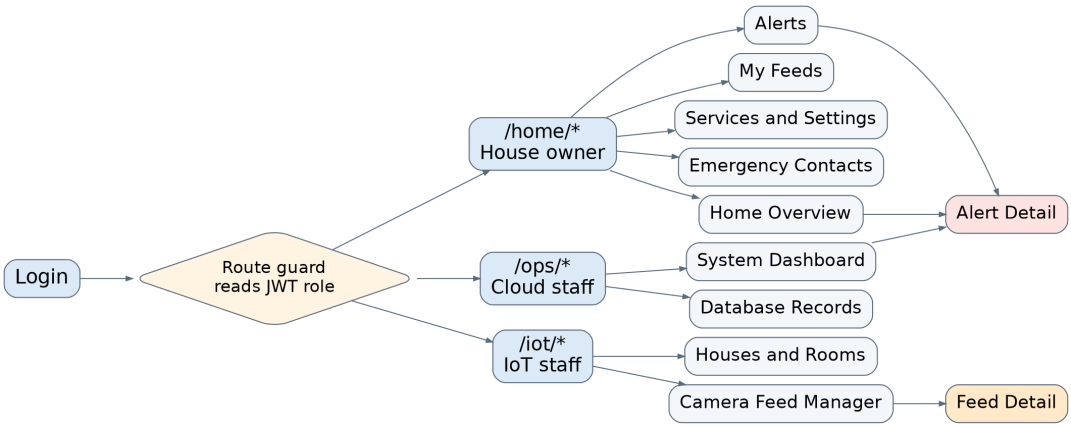



**Figure 5:** Navigation flow: one branch at login into three role workspaces and twelve pages, including the two detail frames

### 3.1.3 UI operation behaviour

Five operations cover the path from sign-in to a configured house.

| Operation | Role | UI steps | Result |
|---|---|---|---|
| Log in and role redirect | all | Enter email and password, then Sign in | The role claim decides the landing page: Home Overview, Camera Feed Manager or System Dashboard |
| Acknowledge an alert | owner | Open the alert from the Home Overview feed or the Alerts table, then Acknowledge | The status chip turns `acknowledged`, the timeline gains a row naming the actor, and the open-alert count drops by one |
| Mark a false positive | owner | Alert Detail, Mark false positive, pick a reason, confirm | The alert leaves the live feed and is recorded as `false_positive` with the reason |
| Add, configure and enable a camera feed | IoT | Camera Feed Manager, Add feed, set name, media kind, house, room, capture interval and sensitivity, Create, then Enable | The row appears as `provisioned` and turns `active` after the first clip arrives; Enable stays disabled until a room is assigned |
| Enable a detection service and set quiet hours | owner | Services and Settings, toggle a service, set quiet hours and sensitivity, Save | Save is enabled only after a change; the next detection uses the new settings, and critical alerts ignore quiet hours |

## 3.2 Hardware Interfaces

The product has no direct hardware interface. Camera feeds are software-registered media sources, and clips reach the platform as uploaded audio or video files or as a phone-camera capture sent over HTTPS. Because no physical device, gateway or firmware is part of the system, connectivity is recorded as a feed status (`connectivity_status`, `network_label`) rather than measured from a network interface.

## 3.3 Software Interfaces

The API service connects to five software components, all within the EC2 instance except the object store and the notification service.

| Component | Version | Data exchanged and purpose |
|---|---|---|
| PostgreSQL | 16 | Reads and writes the relational records (tenants, users, houses, rooms, camera feeds, rules, alerts, notifications) over TCP 5432; every query is scoped by tenant |
| MongoDB | 7 | Reads and writes clip records, AI detections, learned activity patterns and daily statistics over TCP 27017 |
| Amazon S3 | current API | Stores and retrieves clip bytes over HTTPS, authenticated by the EC2 instance's IAM role with no static keys |
| Amazon SNS | current API | Publishes alert email to a house's emergency contacts over HTTPS, authenticated by the instance's IAM role |
| AI inference library | pre-trained audio model (TensorFlow) | Receives a resampled 16 kHz mono clip, or the audio track of a video clip, and returns ranked source classes with confidence, which C3 maps to home labels |

## 3.4 Communications Interfaces

The front-end components and the back-end cloud server communicate over three channels, all on HTTPS port 443 with TLS 1.2 or later.

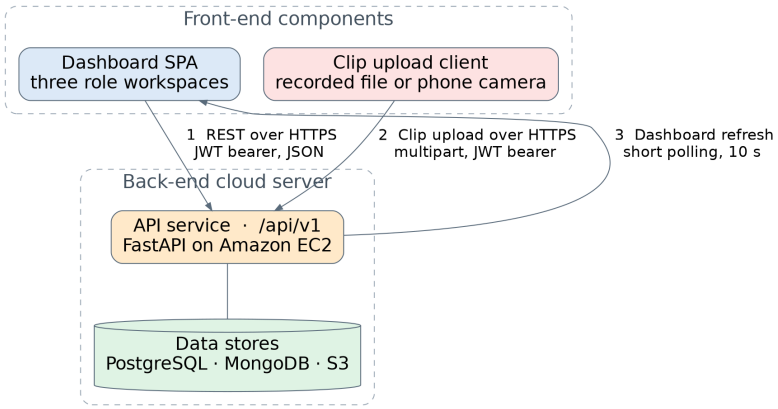



**Figure 6:** Communication channels between the front-end components and the back-end cloud server

| # | Channel | Carries | Reliability |
|---|---|---|---|
| 1 | REST over HTTPS, JSON | All commands and queries under `/api/v1` from the dashboard | Idempotent reads; writes return 409 on a conflicting state so the client reloads the record |
| 2 | Clip upload over HTTPS, multipart | The recorded clip plus its feed identifier and capture time | The clip identifier is client-supplied, so a repeated upload returns the stored record instead of creating a second one |
| 3 | Dashboard refresh by short polling | New and updated alerts and feed status, every 10 seconds | A missed poll is corrected by the next one; the dashboard always renders from the last successful response |

### 3.4.1 Authentication

A successful login returns a signed token carrying the subject, role, tenant and the houses the caller may read. Every protected endpoint re-checks that scope on the server before it touches the database, so the browser never decides access. The dashboard stores the token in memory and attaches it as a bearer header. When the token expires the API returns 401 and the dashboard returns the user to the login page.

### 3.4.2 Message formats

Three formats cover every exchange. All error responses use one envelope, so the dashboard maps field errors and conflicts in one place.

```
// 1  Request                      // 2  List response            // 3  Error envelope
POST /api/v1/alert-rules          { "items": [ { ... } ],        { "error": {
Authorization: Bearer <token>       "page": 1,                     "code": "invalid_state",
Content-Type: application/json      "page_size": 25,               "message": "Alert is already resolved",
{ "alert_type_code": "distress",    "total": 37 }                  "details": { "status": "resolved" } } }
  "trigger_labels": ["scream"],
  "min_confidence": 0.80,
  "room_id": null, "is_enabled": true }
```

### 3.4.3 Component API interactions

Fourteen endpoints carry the integrated demo. Each is owned by exactly one component; a caller from another component uses the owner's API rather than its tables. Classification runs asynchronously: C3's inference worker atomically claims the next `media_events` document with status `queued` (setting it to `processing`), so the collection itself is the work queue. After storing the detection, the worker calls C2's internal `POST /alerts/evaluate`. `POST /inference/classify` remains for cloud staff to re-run classification on demand.

| Owner | Endpoint | Purpose |
|---|---|---|
| C1 | `POST /houses` | Create a house inside the caller's tenant |
| C1 | `POST /feeds` | Register a camera feed |
| C1 | `PUT /feeds/{id}/config` | Save a new configuration version and connectivity status |
| C1 | `POST /ingest/clip` | Upload a clip; creates the clip record as `queued` |
| C2 | `GET /alerts` | List alerts filtered by status, severity, house and period |
| C2 | `POST /alert-rules` | Create an alert rule for a house or for all houses |
| C2 | `POST /alerts/{id}/acknowledge` | Move an alert to `acknowledged` |
| C2 | `POST /alerts/{id}/resolve` | Move an alert to `resolved` with a note |
| C2 | `POST /alerts/evaluate` | Internal: evaluate a stored detection against the house's rules; called by the C3 inference worker |
| C2 | `GET /alert-view/{token}` | Read-only view of one alert for an external recipient; the signed token expires and needs no login |
| C3 | `POST /inference/classify` | Re-run classification of a clip on demand and persist the detection |
| C3 | `GET /records/{entity}` | Paginated, permission-scoped record browsing |
| C4 | `POST /auth/login` | Authenticate and issue the signed token |
| C4 | `GET /dashboard/summary` | Statistics for the selected house and time range |

# 4 System Features

This section organises the functional requirements by system feature, where each feature is one of the four components. The use-case model below spans all four; the per-component subsections then list the detailed functional requirements, and Section 4.5 gives the end-to-end process that ties the features together.

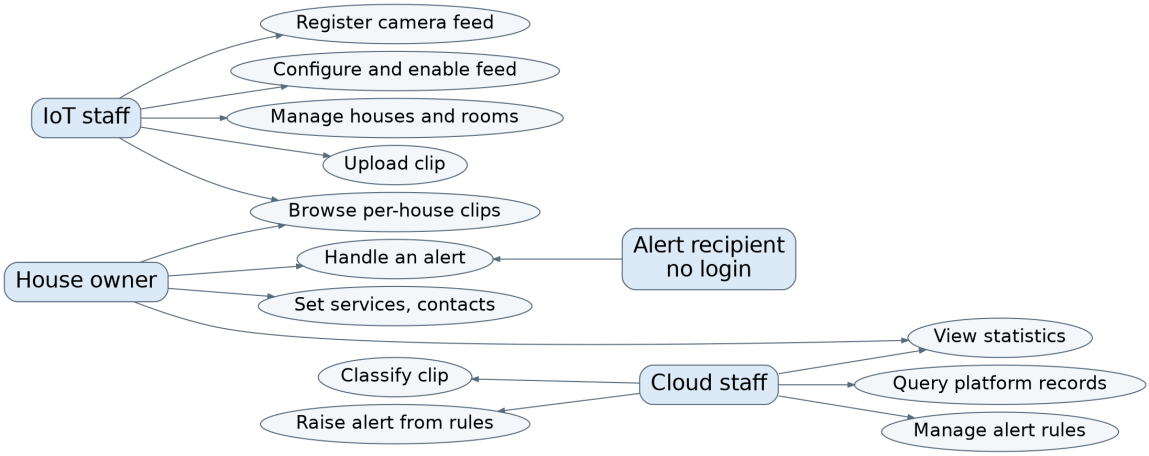



**Figure 7:** Use-case model: four actors and twelve use cases across components C1–C4

| ID | Use case | Primary actor | Main flow | Outcome |
|---|---|---|---|---|
| UC-01 | Register and enable a camera feed (C1) | `iot_staff` | Feed Manager → Add → set name, media kind and configuration → assign house and room → Enable | Feed `provisioned`, then enabled and accepting clips |
| UC-02 | Upload a clip (C1) | `iot_staff` | `POST /ingest/clip` with the feed id and the recorded file | Clip record created with status `queued`; bytes stored in the media store |
| UC-03 | Classify a clip (C3) | `cloud_staff`, system | Load the active model → resample to 16 kHz mono → classify → map source classes to home labels → persist the detection | Labels, confidence and model version stored; clip record set to `analyzed` |

| ID | Use case | Primary actor | Main flow | Outcome |
|---|---|---|---|---|
| UC-04 | Evaluate rules and raise an alert (C2) | system (rules set by `cloud_staff`) | Load the house's enabled rules → match label, minimum confidence, room and service settings → insert the alert → email contacts in priority order | Alert `open` on the dashboard; contacts emailed a signed alert link |
| UC-05 | Handle an alert (C2) | `house_owner` | Open Alert Detail (clip, labels, timeline) → Acknowledge → Resolve or Mark false positive | Status and history row saved |
| UC-06 | Browse and query platform records (C3) | `cloud_staff` | Database Records → pick an entity → filter by house and date → page results; run the integrity report | Permission-scoped record pages; orphan references listed |
| UC-07 | Configure services and contacts (C4) | `house_owner` | Services and Settings → toggle detection services, quiet hours and sensitivity → Emergency Contacts → order the list | Settings and contacts updated; rules apply them from the next detection |
| UC-08 | View platform statistics (C4) | `cloud_staff`, `house_owner` | System Dashboard or Home Overview → pick a house and time range | Alert, detection and feed statistics for the selected scope |
| UC-09 | Manage houses and rooms (C1) | `iot_staff` | Houses and Rooms → add, edit or delete a house or room | House and room records updated in the tenant |
| UC-10 | Browse per-house clip records (C1) | `house_owner`, `iot_staff` | Clip Library → filter by room, feed and date → play a clip | Clip records and labels for one house |
| UC-11 | Manage alert rules (C2) | `cloud_staff` | System Dashboard → Edit rules → set type, severity, trigger labels, minimum confidence and room | Rule saved; applies from the next detection |
| UC-12 | Configure a feed (C1) | `iot_staff` | Feed Detail → change capture interval, sensitivity or network label → Save config, or Restore version | New configuration version saved |

Each member details the use cases of their own component with UML in Deliverable #2.

## 4.1 C1 Home-Based IoT Device Manager

**Description and priority.** C1 registers and configures camera feeds, models houses and rooms, ingests clips, and manages each house's clip records. It is the entry point of the clip-to-alert chain, so its requirements are high priority.

| ID | Requirement | Priority |
|---|---|---|
| FR-C1-01 | The system shall create, update and delete houses and rooms inside the caller's tenant. | Must |
| FR-C1-02 | The system shall register a camera feed with a name, a media kind of `audio` or `video`, and a capture configuration. | Must |
| FR-C1-03 | The system shall update and delete a camera feed and list feeds filtered by house, room, media kind and status. | Must |
| FR-C1-04 | The system shall save each configuration change as a new version holding capture interval, sensitivity and network label, shall restore an earlier version on request, and shall record the feed's connectivity status (`online`, `offline` or `unknown`). | Must |
| FR-C1-05 | The system shall assign a camera feed to exactly one house and one room, and shall record and browse the clip records belonging to each house. | Must |

## 4.2 C2 Alert Tracking and Monitor

**Description and priority.** C2 defines alert rules, evaluates each detection, raises and tracks alerts through their lifecycle, and notifies emergency contacts. Raising and handling alerts is the product's core purpose, so these requirements are high priority.

| ID | Requirement | Priority |
|---|---|---|
| FR-C2-01 | The system shall create, update and delete alert rules per house: alert type, severity, trigger labels, minimum confidence and room. | Must |
| FR-C2-02 | The system shall evaluate each detection against the house's enabled rules and service settings and create an `open` alert on a match. | Must |
| FR-C2-03 | The system shall notify the house's emergency contacts in priority order by email through Amazon SNS, each email carrying a signed, expiring link to a read-only view of the alert; shall show the alert in the in-app inbox of signed-in users of the house; and shall record each notification. | Must |
| FR-C2-04 | The system shall allow only the transitions `open` → `acknowledged`, `acknowledged` → `resolved`, and `open` or `acknowledged` → `false_positive`, and shall reject any other change with HTTP 409. | Must |
| FR-C2-05 | The system shall record every status change with the actor, the time and an optional note. | Must |
| FR-C2-06 | The system shall report alert counts by type, severity, house and period. | Should |

## 4.3 C3 System Database Management

**Description and priority.** C3 runs the AI inference sub-service, maintains the data dictionary, and provides record browsing, query and integrity services across the platform data. Classification and record services are high priority; integrity reporting is secondary.

| ID | Requirement | Priority |
|---|---|---|
| FR-C3-01 | The system shall classify an uploaded clip with the configured pre-trained model, using the audio track of a video clip, and persist the ranked labels, their confidence and the model version. | Must |
| FR-C3-02 | The system shall maintain a versioned data dictionary of every table, collection, field and owning component. | Must |
| FR-C3-03 | The system shall provide paginated, permission-scoped record browsing across houses, feeds, clips, detections and alerts. | Must |
| FR-C3-04 | The system shall run integrity validation that reports orphan references between the relational and document stores. | Should |
| FR-C3-05 | The system shall provide an idempotent demo-data seed and a database backup job. | Must |
| FR-C3-06 | The system shall aggregate each house's detections into an hourly activity profile per room and day and compute an anomaly score against the learned baseline. | Should |

## 4.4 C4 System Dashboard

**Description and priority.** C4 authenticates users, enforces role-based access control, holds owner service settings and contacts, and presents the three role workspaces and the statistics views. Authentication and access control are high priority because every other feature depends on them.

| ID | Requirement | Priority |
|---|---|---|
| FR-C4-01 | The system shall authenticate users by email and hashed password and issue a signed token carrying the user, role, tenant and accessible houses. | Must |
| FR-C4-02 | The system shall enforce role-based access control for `house_owner`, `iot_staff` and `cloud_staff` on every endpoint and every page. | Must |
| FR-C4-03 | The system shall let an owner enable or disable each detection service and set quiet hours and sensitivity per house. House sensitivity raises or lowers the minimum confidence of every rule for that house; it is separate from a feed's capture sensitivity (FR-C1-04), which sets how quiet a sound the feed keeps. | Must |
| FR-C4-04 | The system shall let an owner add, edit, delete and order emergency contacts with a name, phone and email. | Must |
| FR-C4-05 | The system shall present role-specific dashboards with house, feed, alert and detection statistics for a selected time range. | Must |

### 4.5 System process flow

A clip reaches a closed alert in seven steps. Steps 1 and 2 belong to C1, steps 3 and 4 to C3, steps 5 and 6 to C2, and step 7 to C2 through the C4 dashboard.

1. **Upload.** IoT staff or a phone camera uploads a clip for an enabled feed; C1 validates the feed and writes the clip bytes to the media store.
2. **Record.** C1 creates the clip record with status `queued` and stamps it with the house, room, feed and capture time.
3. **Classify.** C3's inference worker claims the clip from the queue, loads the active pre-trained model, takes the audio track of a video clip, resamples the audio to 16 kHz mono and produces ranked source classes with confidence.
4. **Map and store.** C3 maps source classes to the platform's home labels, stores the detection with the model version, and sets the clip record to `analyzed`.
5. **Evaluate.** The worker calls `POST /alerts/evaluate`; C2 loads the house's enabled rules and service settings and tests the top labels against each rule's trigger labels, minimum confidence and room.
6. **Raise or record no alert.** On a match C2 creates an `open` alert and emails the emergency contacts in priority order through Amazon SNS. On no match it records `no_alert` with the reason, so a quiet result is still auditable.
7. **Close.** The owner acknowledges the alert, then resolves it or marks it a false positive; every transition is written to the alert status history.

**Feed onboarding** takes three steps in C1: register the feed with its name, media kind, house and room; set the capture configuration; enable it. Clip uploads are rejected with HTTP 409 (`feed_not_enabled`) until the feed is enabled.

## 5 Other Nonfunctional Requirements

### 5.1 Performance Requirements

**NFR-01.** For a 10-second clip on the single-instance demo deployment, an alert shall appear on the dashboard within 15 seconds of the upload completing, at the 95th percentile. The budget is at most 3 seconds for classification and rule evaluation plus at most one 10-second dashboard poll (Section 3.4). Classification runs on CPU; if it cannot meet this target, the clip window is shortened and audio is classified before video, as noted in the project risks (Section 8.2).

### 5.2 Safety Requirements

**NFR-02.** Audio shall be converted to sound labels only, with no speech transcription, so a resident's conversations are never stored as text. An owner shall be able to switch off any detection service for their own house. Critical alerts, such as a smoke alarm or a detected distress sound, shall ignore quiet hours, so a safety event is never suppressed by a schedule.

### 5.3 Security Requirements

**NFR-03.** All traffic shall use HTTPS with TLS 1.2 or later. Passwords shall be stored as salted hashes. Every query shall be scoped to the caller's tenant and houses, enforced at the service layer and again by the database row-scoping policy. Cloud staff connect through a separate database role whose policy spans all tenants; every other role is limited to its own tenant. Clip bytes in Amazon S3 and alert email through Amazon SNS shall be reached through the EC2 instance's IAM role, with no static credentials stored on the instance. A signed alert link shall open one alert read-only and shall expire after 24 hours.

### 5.4 Software Quality Attributes

| Attribute | ID | Requirement |
|---|---|---|
| Scalability | NFR-04 | Services shall hold no session state, so instances can be added or removed without redesign; the scaling design is given in Section 7.3. |
| Usability | NFR-05 | The dashboard shall be role-based and responsive, and an alert shall be acknowledgeable in at most two clicks from any page. |
| Deployability | NFR-06 | The platform shall deploy to one Amazon EC2 instance with Docker Compose and shall be reproducible from the project repository. |

### 5.5 Business Rules

- A camera feed must be assigned to a room and enabled before it accepts clip uploads; uploads to a disabled feed are rejected with HTTP 409.
- An alert moves only `open` → `acknowledged` → `resolved`, or from `open` or `acknowledged` to `false_positive`; `resolved` and `false_positive` are terminal.
- Emergency contacts are emailed in their stored priority order.
- A house owner sees and acts only on their own houses; cloud staff see all tenants; IoT staff manage feeds for their own tenant.
- Every tenant-owned row carries `tenant_id`, and every request is checked against the caller's tenant before any data is read or written.

## 6 System Architecture Design

This section and the two that follow extend the SRS template to carry the architecture, database, cloud and project-plan content that Deliverable #1 also requires. The template invites added sections for exactly this purpose.

### 6.1 Cloud-Based System Infrastructure

Figure 8 shows the platform in AWS terms, the detailed counterpart to the business-oriented view in Figure 1. Every element the infrastructure diagram must show is present: the user groups, the cloud servers, the data stores and media bucket, the networking path, and the scalability and load-balancing target carried in Section 7.3.

The platform model maps onto this layout as follows.

| Platform model block | HomeAI Cloud element |
|---|---|
| Camera feed sources | Registered camera feeds per room, fed by recorded clips or a phone camera |
| Connectivity network | Home or campus network → internet → HTTPS with TLS 1.2+ terminated at the reverse proxy |
| Home audio and video data store | Amazon S3 media bucket for clip bytes; MongoDB for clip records, detections and learned activity patterns |
| Cloud assistant agent | The AI inference service in C3 and the rule engine in C2 |
| Admin monitor and dashboard | The responsive dashboard in C4 with three role workspaces |

**Infrastructure elements.**

| Element | Service | Purpose |
|---|---|---|
| Compute | Amazon EC2, t3.large | Runs the reverse proxy, the API service, the AI inference service and both databases as Docker containers |
| Relational store | PostgreSQL 16 | Tenants, users, houses, rooms, camera feeds, rules, alerts and notifications |

| Element | Service | Purpose |
|---|---|---|
| Document store | MongoDB 7 | Clip records, AI detections, learned activity patterns and daily statistics |
| Object storage | Amazon S3 | Clip bytes, keyed by tenant, house, feed and date (key format in Section 7.2) |
| Notification | Amazon SNS | Alert email to each house's emergency contacts, one topic per house |
| Entry point | Reverse proxy with TLS | Single HTTPS entry on port 443; routes `/api/v1` to the API service and all other paths to the dashboard build |
| Operations | Amazon CloudWatch, IAM instance role | Logs and metrics; the instance role grants S3 access with no static keys |

**Connectivity and user access.** Only port 443 is open to the internet.

| From → to | Protocol | Authentication | Notes |
|---|---|---|---|
| Browser → reverse proxy | HTTPS 443, TLS 1.2+ | Bearer token | All three roles; REST under `/api/v1` |
| Clip upload client → reverse proxy | HTTPS 443, multipart | Bearer token | `iot_staff` uploads a recorded file or a phone-camera capture |
| API service → PostgreSQL | TCP 5432, container network | Credentials from the environment file | Transactional records; every query scoped by tenant |
| API service → MongoDB | TCP 27017, container network | Database user per service | Clip records, detections, patterns, statistics |
| API service → Amazon S3 | HTTPS 443 | IAM instance role | Clip bytes; no static credentials on the instance |
| API service → Amazon SNS | HTTPS 443 | IAM instance role | Alert email to emergency contacts |

The demo deployment runs every container on one EC2 instance with Docker Compose. Moving to the scaled layout in Section 7.3 changes instance counts and adds the load balancer; the container images and the application code do not change.

## 6.2 Technology Selection

| Layer | Selection | Why |
|---|---|---|
| Frontend | React 18 with Vite, Tailwind CSS, Recharts | One responsive single-page application serving three role workspaces; charts for the statistics dashboard |
| Backend | Python 3.11, FastAPI, Pydantic v2, SQLAlchemy 2 with Alembic | Same language as the AI stack; asynchronous I/O; an OpenAPI contract generated from the code |
| AI model | YAMNet [7], a pre-trained audio event classifier on TensorFlow, 521 source classes | The brief directs us to use the provided pre-trained models; runs fast enough on CPU, so no GPU instance is needed |
| Notification | Amazon SNS email | Managed delivery to contacts who have no account; no mail server to run |
| Relational database | PostgreSQL 16 | Foreign keys, transactional alert status changes, tenant-scoped row access, SQL reporting |
| Document database | MongoDB 7 | Nested label arrays per clip, flexible detection documents, aggregation pipelines for statistics |
| Compute and deployment | One Amazon EC2 instance, Docker Compose, HTTPS reverse proxy | The course requires EC2; one instance keeps the demo reproducible and inside the available credits |

## 6.3 Component-Oriented Function Architecture

HomeAI Cloud partitions into four components at the high level and sixteen sub-functions at the low level. Each sub-function has exactly one owning component. The AI inference service is a platform sub-service inside C3 rather than a fifth component, because it reads the clip store and writes the detection store that C3 owns.

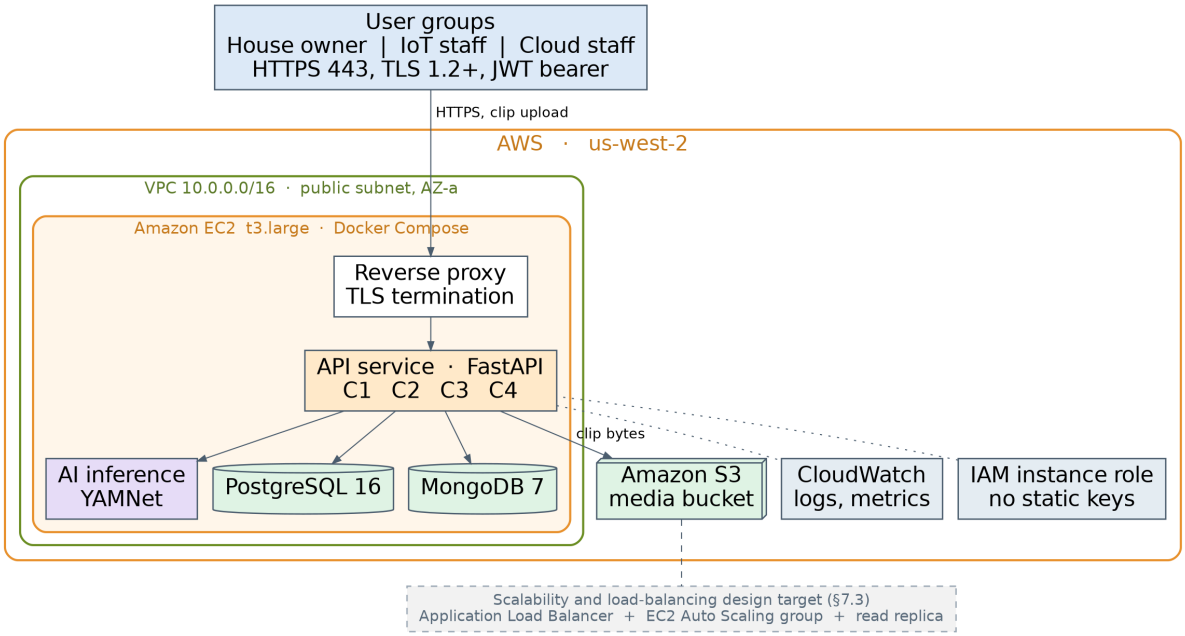



**Figure 8:** Detailed AWS infrastructure for the course demo footprint, with the scale-out design target shown dashed

**Component connectivity and dependencies.** C1, C3 and C2 form the clip-to-alert chain; C4 supplies identity to all three and reads from all three.

| Component | Depends on | Provides to | Data owned |
|---|---|---|---|
| C1 Home-Based IoT Device Manager | C4 for the token and role scope; the media store for clip bytes | C3 clip records; C2 and C4 house, room and feed status | `houses`, `rooms`, `camera_feeds`; `media_events` |
| C2 Alert Tracking and Monitor | C3 detections; C4 service settings and emergency contacts; Amazon SNS for email | Users: alerts, history and statistics; C4 alert feed | `alert_types`, `alert_rules`, `alerts`, `alert_status_history`, `notifications` |
| C3 System Database Management | C1 clip records and clip bytes; C4 for the token and role scope | C2 detections; C4 record pages and the data dictionary | `ml_models`; `ai_detections`, `activity_patterns` |
| C4 System Dashboard | C1–C3 REST APIs for their pages | All components: the signed token with role, tenant and house scope | `tenants`, `users`, `house_service_settings`, `emergency_contacts`; `dashboard_stats_daily` |

**Black-box component views.** Following the black-box modelling practice of [6], each component is specified by what enters it and what leaves it, independently of how it is built, so one member can build a component against the inputs and outputs another provides.

# 7 System Database and Cloud Design

HomeAI Cloud uses two databases and one object store, each matched to one data shape. PostgreSQL holds transactional records that need foreign keys and atomic status changes, MongoDB holds the nested documents produced by clip analysis, and Amazon S3 holds the clip bytes. PostgreSQL issues `tenant_id`, `house_id`, `room_id` and `feed_id` as UUIDs; MongoDB documents and S3 keys copy them, and MongoDB identifiers (`media_id`, `detection_id`) are UUID strings as well, so one identifier type joins all three stores.

## 7.1 Relational Database Design

The schema has 13 tables in three subject areas. Primary keys are UUIDs and history tables use generated identity columns; every timestamp is `TIMESTAMPTZ` in UTC; value sets use `VARCHAR` with a `CHECK` constraint rather than PostgreSQL enumerated types, so a migration can add a value without altering a type. Ten tenant-owned tables carry `tenant_id` and are covered by the row-scoping policy in

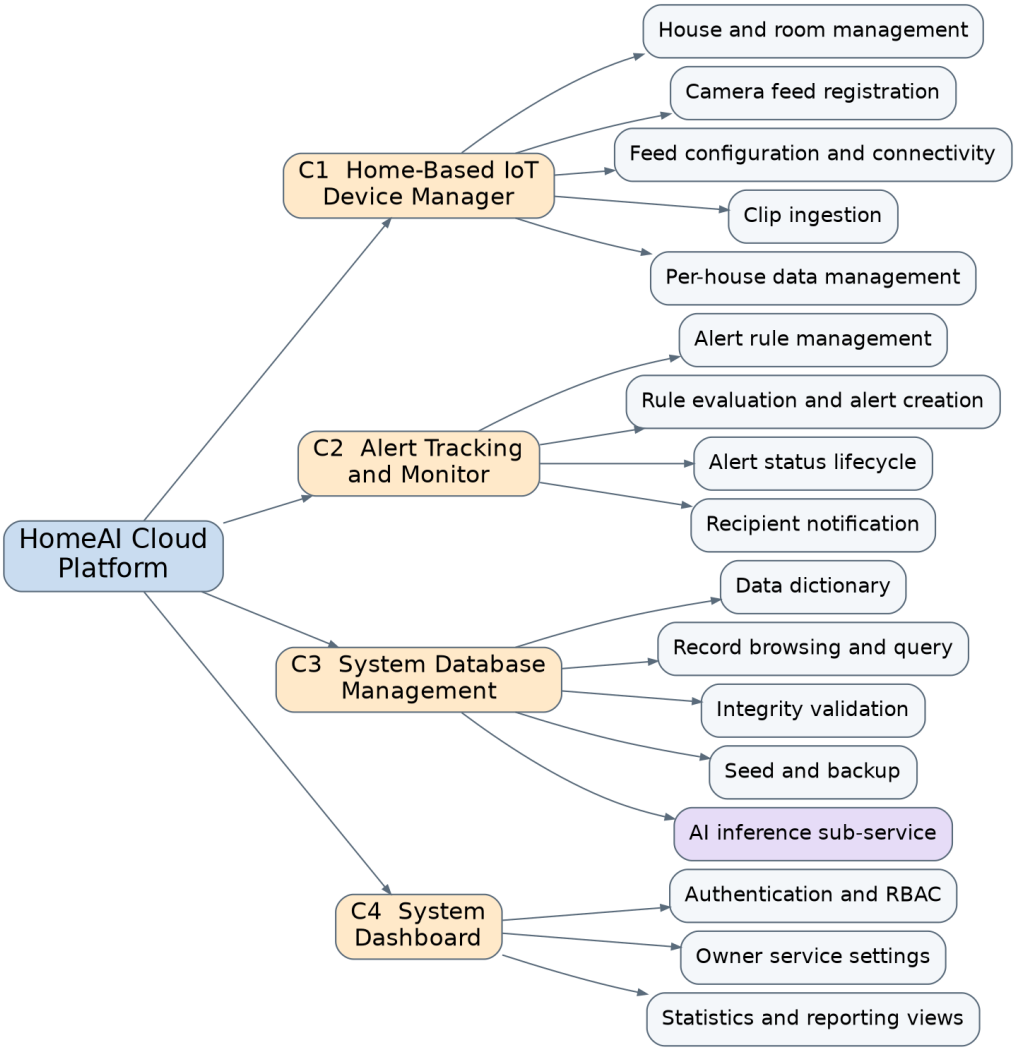



**Figure 9:** Function-oriented tree: four components as the high-level partition and sixteen sub-functions as the low-level partition

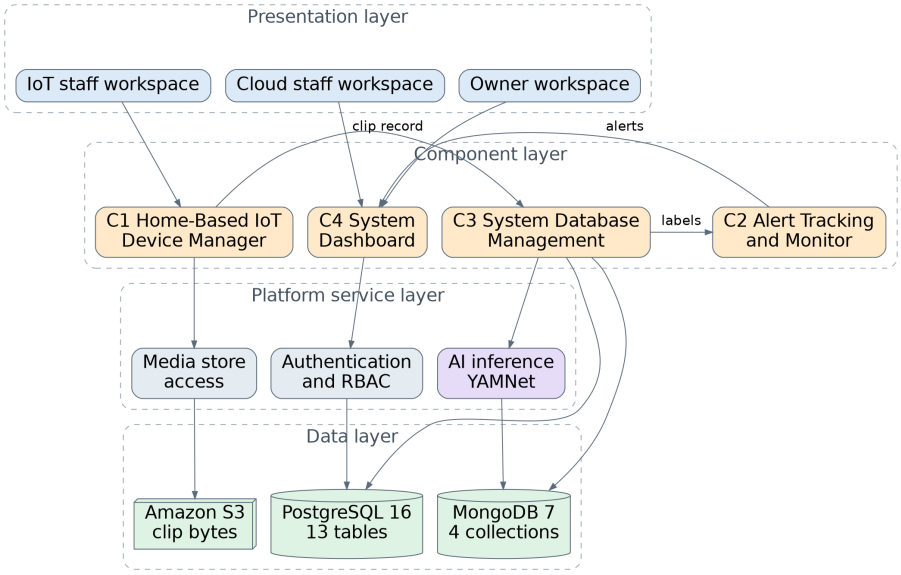



**Figure 10:** System functional block diagram: presentation, component, platform-service and data layers

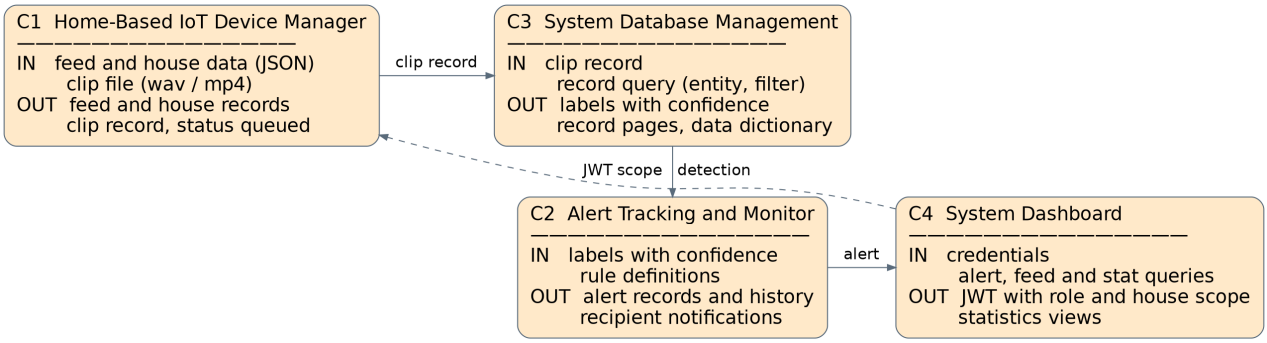



**Figure 11:** Black-box view of the four components: inputs, outputs and the interfaces between them

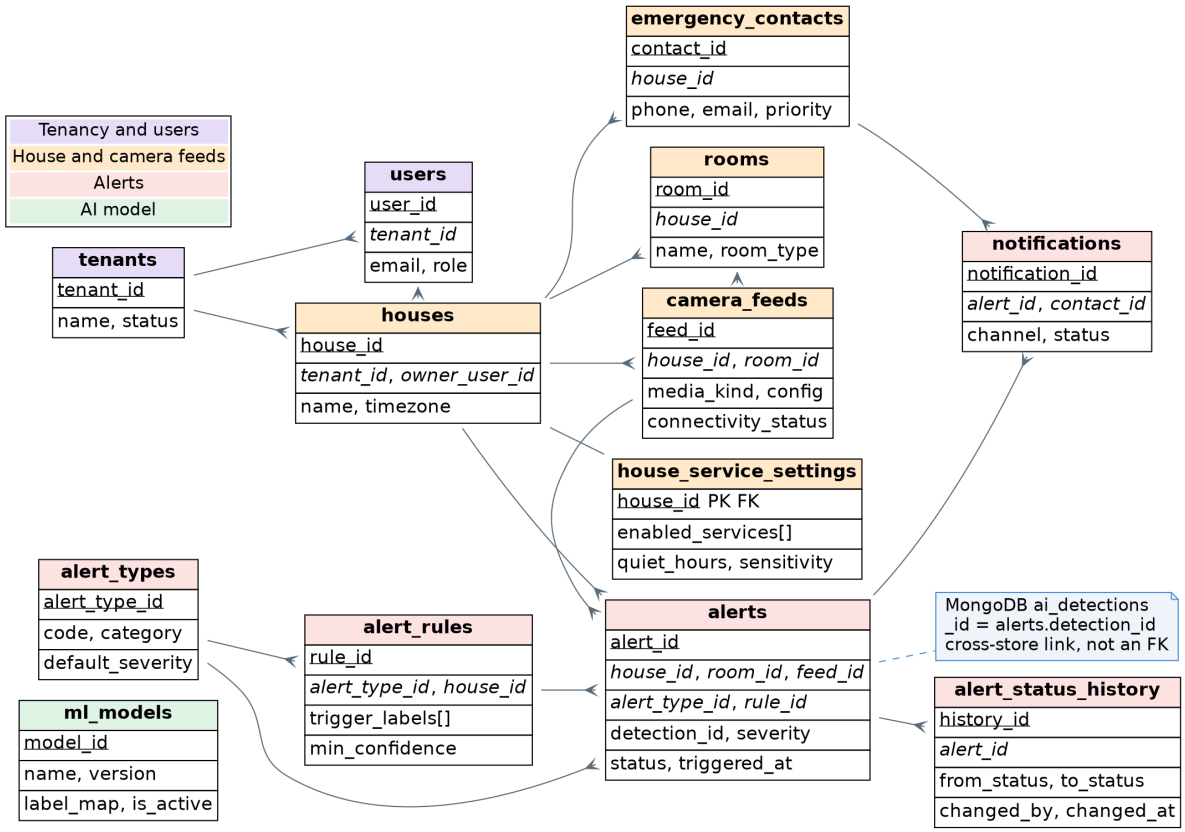



**Figure 12:** Relational entity diagram: 13 tables in three subject areas, with the cross-store link to the MongoDB detection document

| Enumeration | Columns | Values |
|---|---|---|
| Role | `users.role` | `house_owner`, `iot_staff`, `cloud_staff` |
| Feed status | `camera_feeds.status` | `provisioned`, `active`, `inactive`, `error` (`is_enabled` is separate) |
| Connectivity | `camera_feeds.connectivity_status` | `online`, `offline`, `unknown` |
| Notification channel | `notifications.channel` | `in_app`, `email` |
| Alert status | `alerts.status`, both history columns | `open`, `acknowledged`, `resolved`, `false_positive` (last two terminal) |
| Severity | `alerts.severity`, `alert_rules.severity` | `critical`, `high`, `medium`, `low` |
| Alert category | `alert_types.category` | `emergency`, `safety`, `security`, `wellness`, `system` |

## 7.2 NoSQL Database Design

MongoDB 7 holds four collections in database `homeai`, each document carrying `tenant_id` and `house_id` so queries are scoped as on the relational side. These shapes fit documents rather than tables: a clip yields a variable-length ranked label array, and a learned routine is a 24-element hourly profile per room and day. A clip's `media_events.status` moves `queued` → `processing` → `analyzed`, or to `failed` after repeated errors, so the collection doubles as the inference work queue. Clip bytes are stored in Amazon S3 under `tenants/{tenant_id}/houses/{house_id}/feeds/{feed_id}/{yyyy}/{mm}/{dd}/{media_id}.{ext}`.

| Collection | Owner | Purpose | Key fields and indexes |
|---|---|---|---|
| `media_events` | C1 | One document per uploaded clip; the source of each classification request | `_id` = `media_id`, `feed_id`, `room_id`, `media_kind`, `s3`, `captured_at`, `status`, `detection_id`; index `{house_id, captured_at}` |
| `ai_detections` | C3 | One analysis result per clip; the evidence behind an alert | `_id` = `detection_id`, `media_id`, `model`, `labels[]`, `top_label`, `top_confidence`, `detected_at`; indexes `{media_id}`, `{house_id, top_label, detected_at}` |
| `activity_patterns` | C3 | The learned daily routine per house, room and day | `_id` = house:room:date, `hours[]`, `baseline`, `anomaly_score`, `last_activity_at`; index `{house_id, room_id, date}` |
| `dashboard_stats_daily` | C4 | Pre-computed daily statistics per house for the dashboards | `_id` = house:date, `alerts`, `detections`, `feeds`; index `{house_id, date}` |

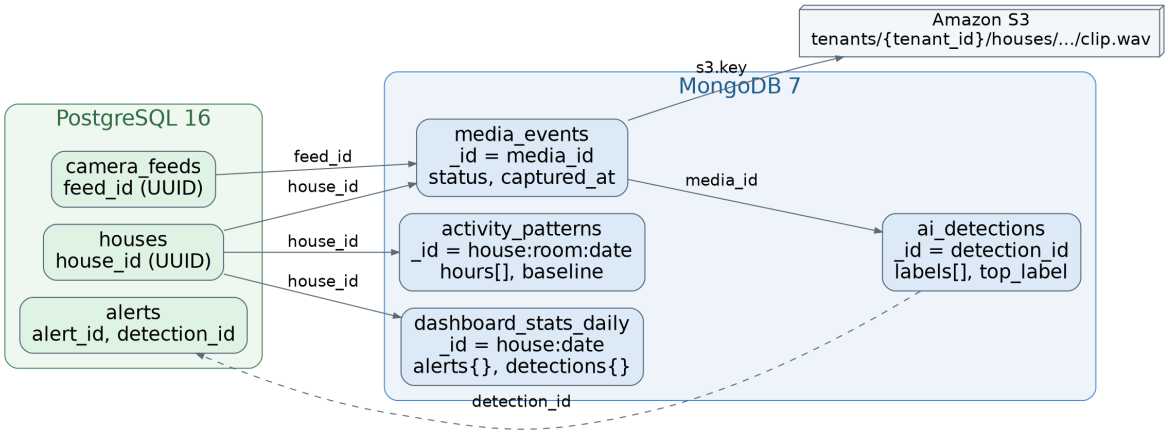



**Figure 13:** MongoDB reference map: four collections joined to the PostgreSQL identifiers and the S3 clip objects

**Sample documents.** Identifiers are shortened for readability; dates are BSON date values shown as ISO 8601 strings. Both samples describe the same clip.

```
// media_events
{
  "_id":        "6f1c2a9e-...",
  "tenant_id":  "0b6e4c1a-...",
  "house_id":   "a3d9f7e2-...",
  "room_id":    "c7e1b2d3-...",
  "feed_id":    "e2f4a6b8-...",
  "media_kind": "audio",
  "source":     "uploaded_recording",
  "s3": { "bucket": "homeai-media",
          "key": "tenants/0b6e/houses/a3d9/
            feeds/e2f4/2026/10/04/6f1c2a9e.wav" },
  "duration_s": 10.0,
  "sample_rate_hz": 16000,
  "captured_at": "2026-10-04T21:14:05Z",
```

```
// ai_detections
{
  "_id":       "6710a3c5-...",
  "media_id":  "6f1c2a9e-...",
  "house_id":  "a3d9f7e2-...",
  "room_id":   "c7e1b2d3-...",
  "media_kind":"audio",
  "model": { "name": "yamnet",
             "version": "1.0.0" },
  "labels": [
    { "label": "scream", "confidence": 0.874,
      "source_class": "Screaming",
      "start_s": 2.88, "end_s": 4.80 },
    { "label": "tv",     "confidence": 0.552 },
    { "label": "speech","confidence": 0.431 } ],
```

```
  "status":      "analyzed",                    "top_label": "scream", "top_confidence": 0.874,
  "detection_id":"6710a3c5-..."                 "inference_ms": 184,
}                                              "detected_at": "2026-10-04T21:14:09Z" }
```

**Aggregation example.** Detections per label for one house over the last 24 hours, used by the Home Overview and the System Dashboard. The first stage uses the `{house_id, top_label, detected_at}` index.

```
db.ai_detections.aggregate([
  { $match:  { house_id: "a3d9f7e2-...",
               detected_at: { $gte: new Date(Date.now() - 24*60*60*1000) } } },
  { $unwind: "$labels" },
  { $match:  { "labels.confidence": { $gte: 0.5 } } },
  { $group:  { _id: "$labels.label", count: { $sum: 1 },
               avg_confidence: { $avg: "$labels.confidence" } } },
  { $sort:   { count: -1 } } ])
```

**Edge-based database design is not applicable to this system.** Clips are uploaded directly to the cloud API from recorded files or a phone camera, so there is no edge agent and no local buffer to design. The relational design in Section 7.1 and the NoSQL design in Section 7.2 are the complete database design.

## 7.3 Cloud Design: Scalability, Load Balancing and Multi-Tenancy

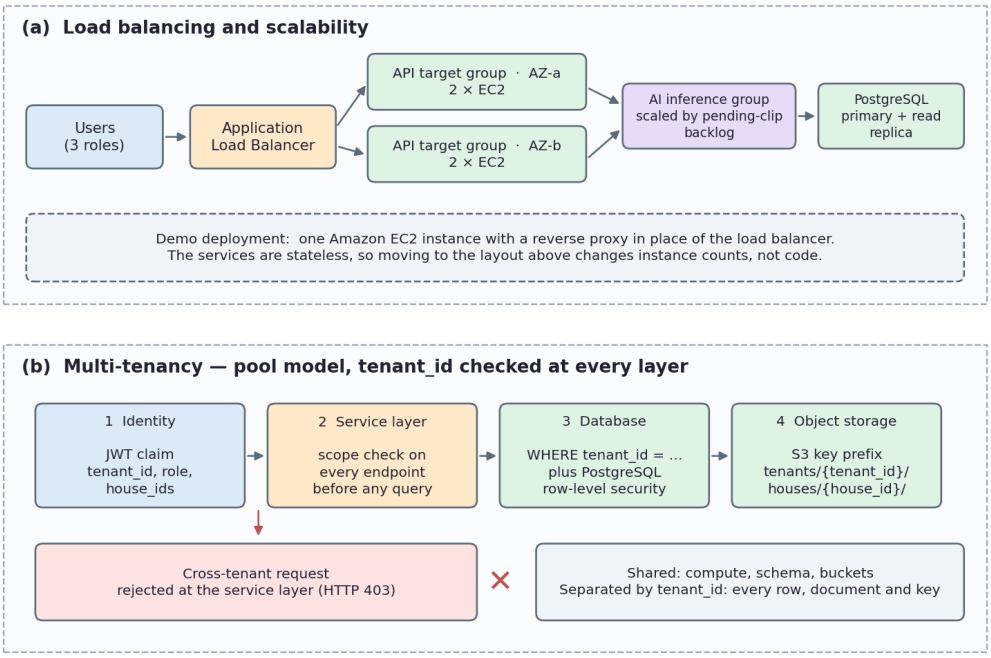



**Figure 14:** Cloud design: load-balanced, horizontally scaled layout and the tenant check applied at each layer

### 7.3.1 Scalability

No service keeps session state. The signed token carries identity, and PostgreSQL, MongoDB and S3 hold all data, so an instance can be added or removed at any time without draining anything but in-flight requests. Each tier therefore scales on its own metric.

| Tier | Scaling mechanism | Trigger | Notes |
|---|---|---|---|
| Web and API | EC2 Auto Scaling group behind the load balancer; every instance runs the same image | Average CPU across the group | Stateless; any node serves any request |
| AI inference | Separate worker process, scaled independently of the API | Pending-clip backlog per worker | A clip left unfinished by a stopped worker returns to `queued` and is picked up again |
| PostgreSQL | Vertical resize for writes; a read replica for dashboard and report reads | Primary CPU sustained high | Writes go to the primary; statistics read from the replica |
| MongoDB | Replica set; hashed shard on `house_id` when the detection volume grows | Storage and CPU on the primary | A house stays on one shard, so per-house queries hit one node |

**Design target against demo footprint.** The demo runs one house with four to six camera feeds on a single EC2 instance, which is the scope the project brief sets. The design target is a care community of many houses, the same community scale as the green energy AI cloud [4], reached by raising instance counts in the two groups above rather than by changing code. The two scaling triggers are average CPU for the API tier and pending-clip backlog for the inference tier; both are measured in Deliverable #3 and reported in the final package rather than estimated here.

### 7.3.2 Load Balancing

One internet-facing Application Load Balancer spans two availability zones, terminates TLS and routes by path prefix to the API and dashboard target groups. Health checks against `GET /health` remove an unhealthy instance from rotation; target groups use least-outstanding-requests so a slow classification request does not queue behind one node; and a deregistration delay drains in-flight calls during scale-in. The demo deploys a single EC2 instance with a reverse proxy in place of the load balancer; the application is stateless, so moving to the load-balanced layout changes instance counts and not code.

### 7.3.3 Multi-Tenancy

HomeAI Cloud uses the pool model: tenants, which are senior-care communities, share the services, the schema and the media bucket, and `tenant_id` is checked at every layer a request passes. A missing filter in application code is still caught one layer down, because the database policy and the object-key prefix enforce the same boundary independently.

| Resource | Shared across tenants | Separated per tenant |
|---|---|---|
| Compute | The API and inference instances | Every request carries the token's `tenant_id`; the service layer rejects a mismatch with HTTP 403 |
| PostgreSQL | One database and one schema | `tenant_id` on ten tables plus a row-scoping policy enforced by the database role the services connect as; cloud-staff requests use a separate role whose policy spans all tenants |
| MongoDB | All four collections | Every document carries `tenant_id` and `house_id`; every query filters on both |
| Amazon SNS | The notification service | One topic per house; only that house's contacts are subscribed |
| Amazon S3 | One media bucket | Key prefix `tenants/{tenant_id}/houses/{house_id}/` |
| Alert rules | Platform default rules written by `cloud_staff` | Per-house overrides of trigger labels, minimum confidence and enablement |

# 8 Project Development Plan and Schedule

Five milestones remain: Deliverable #1, the system design document, on 4 October 2026; Deliverable #2, component detailed design, on 16 October; Deliverable #3, component implementation, on 6 November; Deliverable #4, the integrated demo, on 2 December; and Deliverable #5, the final package, on 16 December. Deliverable #0, project selection and team building, closed on 13 September. Deliverables #2 and #3 are individual; #1, #4 and #5 are team deliverables.

## 8.1 Project Team and Roles

Each member owns one of the four components named in the project brief, end to end: its user interface pages, back-end APIs, data structures, UML detailed design, implementation, tests and demo video.

| Member | Component | Main responsibilities | Cross-cutting role |
|---|---|---|---|
| Member 1 | C1 Home-Based IoT Device Manager | Houses, rooms, camera feeds, configuration versions, connectivity status, clip ingestion, and the per-house clip data management subcomponent | Clip library and demo data |
| Member 2 | C3 System Database Management | Schema and migrations, data dictionary, record browsing and query services, integrity validation, seed and backup, and the AI inference sub-service | Database administration |
| Member 3 | C2 Alert Tracking and Monitor | Alert types and rules, rule evaluation, alert lifecycle and history, emergency-contact email through Amazon SNS, the signed alert view and the notification inbox | Integration testing and QA |
| Member 4 | C4 System Dashboard | Authentication, role-based access control, owner service settings and contacts, the three role workspaces and the statistics views | AWS and EC2 deployment |

**Working practices.** Following Internet-based team development practice [5]: one GitHub repository with a protected `main` branch and one reviewing member per pull request; one team sync each week walking the task board and the critical path; the API contract frozen at Deliverable #2, after which a change needs the agreement of the consuming component's owner.

## 8.2 Project Schedule

The plan has 15 tasks. Durations are calendar days given as optimistic ($o$), most likely ($m$) and pessimistic ($p$); the expected duration and variance follow the PERT formulas

$$t_e = \frac{o + 4m + p}{6} \qquad \sigma^2 = \left(\frac{p - o}{6}\right)^2 .$$

The critical path runs T1 → T2 → T3 → T4 → T6 → T8 → T12 → T13 → T14 → T15 and spans 2 September to 16 December. C3 is the only component on it, because the AI inference sub-service has the least certain duration of the four component builds. Summing the critical tasks gives an expected duration of 80.8 days with a variance of 4.97 ($\sigma \approx 2.2$ days). The planned window is 106 calendar days, so the schedule holds about 25 days of buffer, placed mainly before Deliverables #4 and #5.

| ID | Task | Owner | Pred. | $o/m/p$ | $t_e$ | Planned dates | Crit. |
|---|---|---|---|---|---|---|---|
| T1 | Project selection and team building (D0) | All | — | 10/12/14 | 12.0 | Sep 2 – Sep 13 | Yes |
| T2 | Requirements analysis and SRS | All | T1 | 5/7/9 | 7.0 | Sep 14 – Sep 20 | Yes |
| T3 | Architecture and C1–C4 partition | All | T2 | 4/5/6 | 5.0 | Sep 21 – Sep 25 | Yes |

| ID | Task | Owner | Pred. | $o/m/p$ | $t_e$ | Planned dates | Crit. |
|---|---|---|---|---|---|---|---|
| T4 | Database design, relational and document | M2 | T3 | 4/6/8 | 6.0 | Sep 26 – Oct 1 | Yes |
| T5 | Dashboard UI and cloud design | M4 | T3 | 3/5/7 | 5.0 | Sep 26 – Sep 30 | No |
| T6 | D1 assembly and review | All | T4, T5 | 2/3/4 | 3.0 | Oct 2 – Oct 4 | Yes |
| T7 | C1 detailed design with UML | M1 | T6 | 8/10/12 | 10.0 | Oct 5 – Oct 14 | No |
| T8 | C3 detailed design with UML | M2 | T6 | 9/12/15 | 12.0 | Oct 5 – Oct 16 | Yes |
| T9 | C2 detailed design with UML | M3 | T6 | 8/11/13 | 10.8 | Oct 5 – Oct 15 | No |
| T10 | C4 detailed design with UML | M4 | T6 | 9/11/14 | 11.2 | Oct 5 – Oct 16 | No |
| T11 | C1, C2 and C4 implementation | M1, M3, M4 | T7, T9, T10 | 12/14/17 | 14.2 | Oct 17 – Nov 6 | No |
| T12 | C3 implementation, AI and database services | M2 | T8 | 13/16/20 | 16.2 | Oct 17 – Nov 6 | Yes |
| T13 | Integration and EC2 deployment | All | T11, T12 | 7/9/12 | 9.2 | Nov 9 – Nov 20 | Yes |
| T14 | D4 demo, presentation and report | All | T13 | 4/5/7 | 5.2 | Nov 23 – Dec 2 | Yes |
| T15 | D5 final package | All | T14 | 4/5/6 | 5.0 | Dec 3 – Dec 16 | Yes |

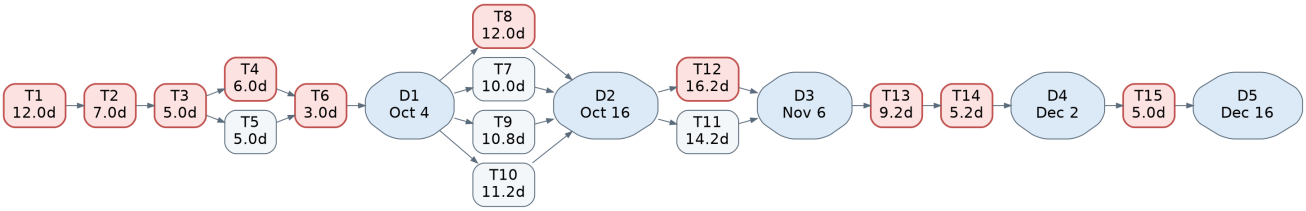



**Figure 15:** PERT network: 15 tasks and 5 milestones, with the critical path highlighted

**Risks.** Three risks can move a milestone; each has an owner and a planned response.

| Risk | Schedule impact | Mitigation | Owner |
|---|---|---|---|
| Classification on CPU is slower than the alert target | T12 slips, moving D3 and D4 | Benchmark the model during T8; if slow, shorten the clip window, cache the loaded model in the worker process, and classify audio before video | Member 2 |
| AWS spend exceeds the available credits | The environment is stopped before D4 | Keep the one-instance demo footprint, stop the instance outside test sessions, and set a billing alarm | Member 4 |
| Components disagree at integration | T13 overruns, moving D4 | Freeze the OpenAPI contract at D2 and run contract tests in continuous integration from 17 October | Member 3 |

# Appendix A: Glossary

| Term | Meaning |
|---|---|
| Tenant | A senior-care community that holds an account on the platform; the top level of data isolation. |
| House | One senior's home within a tenant, owned by a house owner. |
| Room | A named space inside a house to which camera feeds are assigned. |
| Camera feed | A software-registered media source for one room; audio or video, fed by recorded clips or a phone camera. |
| Clip | One uploaded audio or video recording from a feed, stored in Amazon S3 and recorded in `media_events`. |
| Detection | The AI result for one clip: ranked home labels with confidence, stored in `ai_detections`. |
| Label | A home sound category such as `smoke_alarm`, `scream` or `speech`, mapped from the model's source classes. |
| Alert rule | A per-house rule that turns a detection into an alert when its trigger labels, minimum confidence and room match. |

| Term | Meaning |
|---|---|
| Alert | An event raised from a matching detection, moving through `open`, `acknowledged` and `resolved` or `false_positive`. |
| RBAC | Role-based access control; the check of a caller's role on every endpoint and page. |
| JWT | The signed bearer token issued at login, carrying the user, role, tenant and accessible houses. |
| Pool model | A multi-tenancy model in which tenants share compute, schema and bucket, separated by `tenant_id` at every layer. |
| YAMNet | A pre-trained audio event classifier with 521 source classes, used as the AI inference model. |
| Amazon SNS | Amazon Simple Notification Service; sends alert email to emergency contacts who hold no account. |
| Signed alert link | A time-limited URL in an alert email that opens one alert read-only, with no login. |

# Appendix B: Analysis Models

The analysis models for this system are presented in place rather than collected here, so each sits beside the text it supports:

- **Use-case model** — Figure 7, Section 4.
- **Function-oriented tree** — Figure 9, Section 6.3.
- **Functional block diagram** — Figure 10, Section 6.3.
- **Black-box component view** — Figure 11, Section 6.3.
- **Entity-relationship diagram** — Figure 12, Section 7.1.
- **MongoDB reference map** — Figure 13, Section 7.2.
- **System process flow** — Section 4.5.
- **PERT network** — Figure 15, Section 8.2.

# Appendix C: To Be Determined List

| ID | Item | Owner | Resolved by |
|---|---|---|---|
| TBD-1 | Final map from YAMNet source classes to home labels | C3 (Member 2) | Deliverable #2 |
| TBD-2 | Default minimum confidence per alert type | C2 (Member 3) | Deliverable #3 |
| TBD-3 | Measured upload-to-dashboard latency against NFR-01 | C3 (Member 2) | Deliverable #3 |
| TBD-4 | Signed alert-link expiry, 24 hours proposed (NFR-03) | C2 (Member 3) | Deliverable #2 |
| TBD-5 | Scaling trigger thresholds for the API and inference tiers (Section 7.3) | C4 (Member 4) | Deliverable #3 |